Claude Code 深度体验报告

> 体验时间：2026-10 | 体验者：吕滢滢 | 广东金融学院 金融科技 说明：产品功能与价格以官网为准；本文结合公开资料与实际使用，重点分析产品策略与体验，不构成购买建议。

一、产品定位

- **一句话定位**：Anthropic 的 agentic coding 工具——不是「更聪明的代码补全」，而是把 Claude 放进真实代码库，执行「读仓库 → 做计划 → 改多文件 → 跑测试/命令 → 根据结果继续改」的完整工程循环。
- **目标用户画像**：
- 核心圈层：后端/全栈工程师、独立开发者、技术型创始人——每天在终端和 IDE 里工作，愿意用命令行换自动化。
- 第二圈层：数据/量化研究者、技术型产品经理——需要跨文件改代码、跑实验、做原型，但不一定有完整工程团队。
- 第三圈层：希望把 AI 接入团队流程的技术负责人——看重权限、审计、可复现，而不只是「补全快不快」。
- **核心使用场景（3 个）**：
- 跨文件重构：把一个模块从同步改异步、统一错误处理、迁移 API，涉及十几个文件时，让 Agent 自己找调用点、改代码、跑测试。
- 修 bug / 读陌生代码库：给它一个失败测试或 issue，让它定位、提出假设、修改、验证，而不是自己一行行 grep。
- 把重复工程流程脚本化：用 Skills / Hooks / Subagents 把代码审查、发布检查、数据跑批等流程固化，变成团队可复用的 Agent 工作流。
- **市场位置 & 竞品**：Claude Code 站在「编程 Agent」而不是「AI IDE」的位置。它不跟 Cursor 比谁的内联补全更顺滑，而是比谁能把更长、更复杂的工程任务从头做到尾。核心竞品：OpenAI Codex（云端并行 + ChatGPT 分发）、Cursor（IDE 体验 + 多模型）、GitHub Copilot（GitHub 生态 + 企业采购）、Gemini CLI（Google 生态 + 低价）。
- **收费模式**：没有独立免费版；个人通过 Claude Pro（约 $20/月）或 Max（约 $100/$200/月）使用，团队走 Team / Enterprise；也可以按 API token 计费。重度使用的成本主要取决于任务长度和模型档位。

二、核心功能拆解

功能 A：仓库级任务循环

- **用户流程**：在项目目录启动 → 用自然语言描述任务 → Agent 读取相关文件、给出计划 → 修改多个文件 → 运行测试/命令 → 根据报错继续修 → 最后给出 diff 和总结。
- **交互亮点**：
- 它不是「生成一段代码让你复制」，而是直接在你的工作区里改文件、跑命令，形成一个可验证的闭环。
- 支持 plan mode：先只读、先出计划，用户确认后再动手。对高风险改动，这个「先看计划再执行」的设计比「直接改」更符合工程习惯。
- diff 展示和逐轮总结让用户能随时接管，不是黑盒。
- **待改进点**：
- 任务越长，上下文压缩越容易丢细节；跨十几个文件的重构，仍需要人盯着关键接口。
- 计划质量依赖用户描述；需求模糊时，它会「合理地猜」，猜错后返工成本高。

- 跑测试/命令的权限确认在频繁操作时会打断心流，需要提前配置好允许列表。

功能 B：可编排的定制层（CLAUDE.md / Skills / Subagents / Hooks / MCP）

- **用户流程**：在项目里写 CLAUDE.md 约定规范 → 把常用流程写成 Skill → 用 Subagent 隔离探索/审查任务 → 用 Hook 在文件修改后自动格式化/跑 lint → 通过 MCP 接外部数据源和工具。
- **交互亮点**：
- 这是 Claude Code 最像「平台」而不是「工具」的地方：模型能力是底座，真正形成差异的是这些可复用的工作流资产。
- Subagent 用独立上下文窗口做探索，只把结论带回主会话，避免主上下文被大量搜索结果污染。
- Hooks 是确定性的生命周期脚本，不依赖模型「记得做」——比如每次改完文件自动跑格式化，这比在 prompt 里反复叮嘱可靠。
- **待改进点**：
- 概念多、配置散：CLAUDE.md、Rules、Skills、Subagents、Hooks、MCP、Plugins 各管一摊，新用户很难判断「我该用哪个」。
- 调试成本高：Hook 没触发、Skill 没加载、Subagent 没按预期工作，排查路径不直观。
- 团队共享缺少「开箱即用的最佳实践包」，大量项目仍在重复造轮子。

功能 C：多入口与异步/并行

- **用户流程**：终端里启动一个任务 → 在 IDE 里看 diff → 把长任务放到 Web/云端会话继续跑 → 手机上查看进度 → 需要时把云端会话接回本地终端。
- **交互亮点**：
- 「同一引擎，多入口」是清晰的产品策略：终端是能力最全的参考实现，IDE / 桌面 / Web 各自补足场景。
- Web 会话跑在云端 VM 上，适合「关掉电脑也让任务继续」；对长任务和并行任务很有价值。
- 多会话 / 多 worktree 让用户可以同时推进多个分支，接近「管理一个 Agent 团队」的体验。
- **待改进点**：
- 不同入口的能力不完全一致：Web / cloud sessions 不支持 Terminal CLI、部分 IDE 与某些云厂商路径，CLI-only 能力在其他入口会缺失。
- 多入口带来「任务在哪、状态是什么」的认知负担，缺少一个统一的任务看板。
- 云端会话和本地环境的差异（依赖、密钥、数据）需要用户自己处理。

三、做得好的地方

1. **把「模型能力」封装成「工程流程」** — 产品视角
2. Claude Code 的护城河不只是 Claude 模型，而是它把「读仓库、做计划、改文件、跑测试、看 diff」这套工程动作产品化了。用户买的不是一次回答，而是一个可重复的工作方式。
3. 举个例子：一个跨 10 个文件的重构，传统聊天式 AI 需要用户自己拆任务、复制代码、回来粘贴；Claude Code 可以自己找调用点、改完跑测试、根据报错继续修。这个闭环才是「Agent」和「Chatbot」的分水岭。
4. **定制层让 Agent 从「个人玩具」变成「团队基础设施」** — 产品视角
5. CLAUDE.md 把项目规范写进上下文，Skills 把重复流程沉淀下来，Hooks 把「必须做的事」变成确定性脚本，Subagents 把探索和主任务隔离。四层加起来，解决的是「怎么让 Agent 稳定地按团队方式工作」。
6. 这带来很强的切换成本：团队积累的 Skills/Hooks/规范越多，迁移到另一个 Agent 的成本越高。这比单纯比模型分数更有商业价值。

7. **多入口覆盖了「深度工作」和「异步委派」两种模式** — 用户视角
8. 终端适合深度、连续、需要频繁确认的工作；Web/云端适合把长任务丢出去，回来再看结果；IDE 适合边看 diff 边改。三种模式对应三种工作节奏，而不是把一个界面硬塞给所有人。

坦诚说：Claude Code 的三个硬伤

1. **成本与额度是最大的日常摩擦。** 没有免费版，Pro/Max 的额度还要和 Claude 聊天共享；重度 Agent 任务很容易触发 5 小时/每周上限。按 API 计费虽然灵活，但长上下文 + 多轮工具调用会让账单不可预测。对个人开发者，这是一道真实的心理门槛。
2. **终端优先带来学习曲线和配置复杂度。** 它奖励熟悉命令行、愿意写 CLAUDE.md、会配 Hooks 的用户；对习惯 IDE 图形界面的开发者，前几小时的学习成本明显高于 Cursor/Copilot。功能很强，但「强」本身也是一种使用门槛。
3. **权限与自治之间的平衡还没完全解决。** 沙箱/审批需要用户自己配置；给太少权限，Agent 频繁停下来问；给太多权限，又要承担误改文件、跑错命令的风险。真正让用户放心把长任务交出去，还需要更好的风险分级和可回滚体验。

四、如果我是 PM，我会改进什么

1. **任务级成本预估与额度看板** — 优先级：高
2. 理由：用户在执行大任务前不知道会花多少额度，往往做到一半被限额打断。成本不确定性会直接抑制「把大任务交给 Agent」的行为。
3. 改进方案：启动任务前，根据仓库规模、任务类型、预计工具调用次数给出 token/额度区间；执行中显示已消耗；快接近上限时提示「切换模型/缩小范围/改用 API」。
4. 预期效果：减少因额度中断导致的失败任务，提升重度用户的留存和付费意愿。
5. **按风险分级的权限预设** — 优先级：高
6. 理由：当前权限模型偏向「逐次确认」，高频操作下非常打断心流；但完全放开又让用户不安。
7. 改进方案：提供「只读探索」「改代码 + 跑测试」「允许安装依赖」「允许网络」等风险档位，按任务一键选择；对高风险命令（删除、推送、发布）单独二次确认；所有操作可回滚。
8. 预期效果：降低审批疲劳，同时把「敢不敢放手」变成可配置的产品能力。
9. **从「配置框架」到「开箱即用的团队模板」** — 优先级：中
10. 理由：CLAUDE.md / Skills / Hooks / Subagents 的概念很强，但新用户不知道从哪开始，团队也没有标准模板。
11. 改进方案：首次进入仓库时自动扫描技术栈，生成一份可编辑的 CLAUDE.md 初稿；提供按语言/框架的 Skills/Hooks 模板（如 Python 格式化、前端 lint、数据跑批）；用「推荐配置」而不是「空白编辑器」引导。
12. 预期效果：缩短首次价值时间，让更多用户跨过「配置门槛」进入深度使用。

五、竞品对比简表

维度	Claude Code	Codex	Cursor	GitHub Copilot
产品形态	终端为核心，IDE / 桌面 / Web 多入口	CLI + IDE + 云端 + ChatGPT	AI 原生 IDE	IDE 插件 + GitHub 平台
核心优势	长任务、多文件重构、定制层深	云端并行、ChatGPT 分发、低门槛	内联编辑体验、多模型	GitHub 生态、企业采购
核心劣势	成本/额度、终端学习曲线	计划漂移、计量不透明	Agent 深度弱于专用工具	Agent 能力偏保守

维度	Claude Code	Codex	Cursor	GitHub Copilot
目标用户	重度开发者、技术负责人	ChatGPT 用户、工程团队	广泛开发者	企业开发团队
收费模式	Pro/Max 订阅 + API	随 ChatGPT 订阅 + API	订阅制	个人/企业订阅
一句话	最像「工程搭档」	最像「任务队列」	最像「AI IDE」	最像「企业标配」

六、一个启发

Claude Code 让我重新理解「Agent 产品」的竞争壁垒：模型能力只是入场券，真正的壁垒是 **工作流封装 + 信任设计 + 资产沉淀**。

工作流封装，是把「读仓库、做计划、改文件、跑测试」这些工程动作变成产品默认行为；信任设计，是让用户知道 Agent 做了什么、能回滚、能审计；资产沉淀，是 CLAUDE.md、Skills、Hooks 这些东西越用越多，迁移成本越来越高。

这对 AI 产品经理的启发是：不要只问「我的模型比对手强多少」，要问「用户把任务交给我的产品之后，敢不敢离开、回来时能不能验收」。Agent 产品的核心指标不是「生成速度」，而是「任务完成率 × 用户信任度」。

七、运营视角拆解

拉新：开发者社区口碑 + 开源生态。Claude Code 的增长很像开发者工具：不是投广告，而是靠「一个真实的重构案例」在技术社区里传播；Skills/Plugins 生态让第三方愿意帮它做内容。

激活：首次价值时刻不是「装好」，而是「第一个跨文件任务被成功完成」。因此 onboarding 的关键是帮用户找到一个「足够小、但又能体现 Agent 价值」的任务，而不是一上来就让他配置一堆东西。

留存：CLAUDE.md、Skills、Hooks、Subagents 是天然的留存资产——用户积累越多，迁移成本越高；团队把 Agent 接入 CI/代码审查后，留存从「个人习惯」升级为「组织流程」。

商业化：订阅制 + API 双轨。Pro 降低尝试门槛，Max 承接重度用户，Team/Enterprise 卖给需要权限、审计、统一管理的组织。风险在于额度不透明会把重度用户推向 API 或竞品。

运营痛点：成本预期管理、权限摩擦、配置门槛。运营需要把「强」翻译成「可感知的省时间」，否则用户只记住了账单。